

What is HTTP & HTTPS

HTTP (HyperText Transfer Protocol):

- Used for communication between browser and server.
- Data sent as plain text (not encrypted).

HTTPS (HTTP Secure):

- Adds SSL/TLS encryption for secure data transfer.
- Shows  lock symbol in the browser.
- Protects against man-in-the-middle attacks.

Request-Response Cycle

- Client (browser) sends a request to a server.
- Server processes it and sends a response (HTML, JSON, etc).
- Request: GET /login
- Response: 200 OK + page content.
- Hackers exploit this communication by intercepting or modifying requests/responses.

HTTP Methods (Verbs)

- GET: Retrieve data (visible in URL).
- POST: Send data securely (form submission).
- PUT / PATCH: Update data.
- DELETE: Remove data.
- Understanding these is crucial, many attacks target specific methods (like CSRF using POST).

HTTP Headers

- Extra information sent along with requests/responses.

Examples:

- Content-Type, User-Agent, Cookie, Authorization
- Security headers like X-Frame-Options, Content-Security-Policy (CSP)
- Attackers may manipulate headers to bypass controls.

Cookies & Sessions

- Cookies: Small data stored in browser (used for login sessions, preferences).
- Sessions: Server-side data linked with a user's cookie.
- Risks: Session hijacking, cookie theft via XSS.
- Secure flags to protect: HttpOnly, Secure, SameSite.

Encryption & SSL/TLS

- Converts readable data into unreadable form (ciphertext).
- Ensures Confidentiality, Integrity, Authenticity (CIA triad).
- SSL/TLS makes HTTPS secure.
- Certificates verify server identity.

Authentication vs Authorization

- Authentication: Verifying who you are (login).
- Authorization: Checking what you can do.
- Authentication: Entering username/password.
- Authorization: Only admin can delete users.
- Attackers exploit broken logic here → “Broken Authentication” vulnerability.

Firewall, WAF & Security Layers

- Firewall: Filters network traffic (IP-level).
- WAF (Web Application Firewall): Filters HTTP/HTTPS traffic for malicious patterns (SQLi, XSS).
- Web security works in layers — not one single tool!

Client-Side vs Server-Side

- Client-side: Code that runs in browser (HTML, JS, CSS).
- Server-side: Code that runs on backend (Node, PHP, Python, etc).
- Many attacks exploit client–server trust boundary.

Input Validation & Output Encoding

- Input Validation: Check & sanitize user input before using it.
- Output Encoding: Escape special characters before showing data in browser.
- Core defense against XSS, SQLi, etc.